

Shiv Nadar University Chennai

Name of Student	KR Srinidhi Rajamane	Reg. No. 24011101059
Degree & Branch	B.Tech Artificial Intelligence & Data Science Section A	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026– 27

Experiment 3 Implementation of Convolutional Neural Networks (CNNs) for Image Classification

1. Objective

To understand the working principle of Convolutional Neural Networks by implementing convolution, pooling, feature map visualization, and image classification using PyTorch.

2. Background Theory

A Convolutional Neural Network consists of a Convolution Layer, Activation Layer, Pooling Layer, and Fully Connected Layer.

The convolution operation is

$$Y(i, j) = \sum_m \sum_n X(i + m, j + n) K(m, n)$$

where X is the input image, K is the kernel, and Y is the resulting feature map.

The output feature map size is given by

$$\text{Output Size} = \frac{N - F + 2P}{S} + 1$$

where N is the input size, F is the filter size, P is the padding, and S is the stride.

3. Dataset Description

Dataset: CIFAR-10

Training Images	50,000
Testing Images	10,000
Classes	10
Image Size	$32 \times 32 \times 3$ (RGB)

Class Labels

Plane, Car, Bird, Cat, Deer, Dog, Frog, Horse, Ship, Truck.

4. Experimental Procedure

Task 1: Dataset Exploration

- Load CIFAR-10.
- Display ten sample images.
- Print dataset dimensions.
- Plot class distribution.

Task 2: Convolution Kernel Size Comparison

- Implement a convolution layer.
- Compare kernel sizes 3×3 , 5×5 , 7×7 .
- Record resulting feature map sizes.

Task 3: Stride and Padding Study

- Compare stride = 1 and stride = 2.
- Compare padding = Same and padding = Valid.
- Compute output dimensions using the standard formula.

Task 4: Feature Map Visualization

- Visualize feature maps after the first convolution layer.
- Display at least eight feature maps.

Task 5: Pooling Comparison

- Compare Max Pooling and Average Pooling.
- Compare output size and resulting feature representation.

Task 6: CNN Construction and Training

Input $\rightarrow$ Conv $\rightarrow$ ReLU $\rightarrow$ MaxPool $\rightarrow$ Conv $\rightarrow$ ReLU $\rightarrow$ MaxPool $\rightarrow$ Flatten $\rightarrow$ Dense $\rightarrow$ Softmax

Trained using Optimizer: Adam, Epochs: 20, Batch Size: 32.

Task 7: Model Evaluation

Computed Accuracy, Precision, Recall, F1-score, Confusion Matrix, and Classification Report.

5. Source Code

The complete implementation was carried out in Python using PyTorch, TorchVision, NumPy, Matplotlib, Seaborn and Scikit-learn. The full, executable source code (organized cell-wise, exactly as it was run) is available on GitHub:

https://github.com/Srinidhi-Krishna/Deep-Learning-Lab/blob/main/Lab_3/DL_LAB_3.ipynb

6. Generated Plots with Interpretation

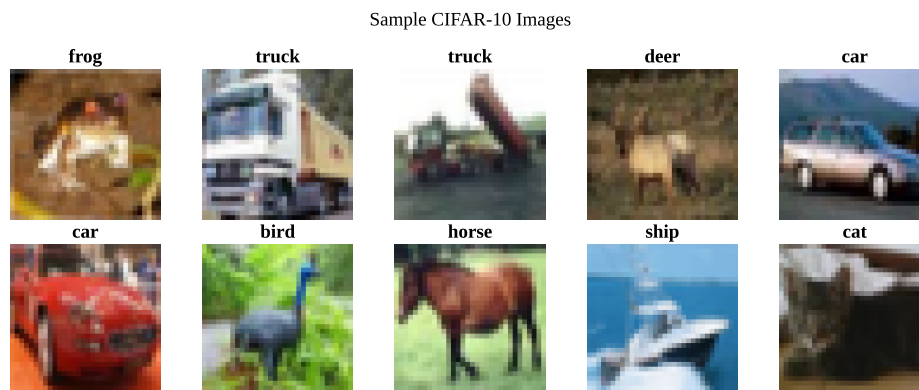


Figure 1: Ten sample images from the CIFAR-10 training set with class labels.

Interpretation: The sample images show varied object poses, backgrounds, and lighting conditions across classes, confirming the dataset's suitability for a robust classification task.

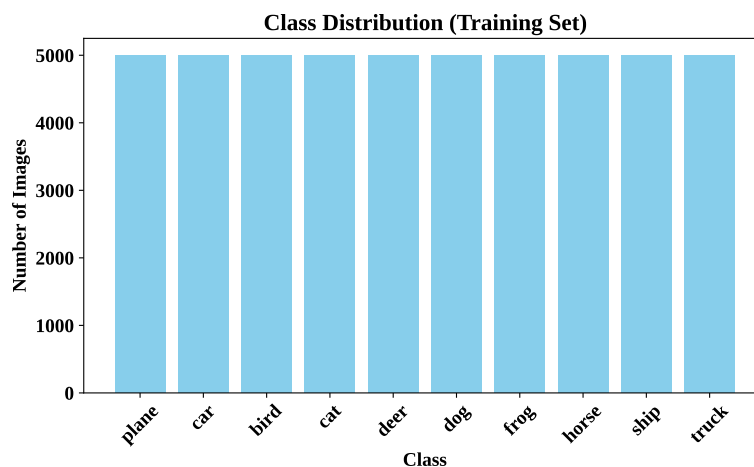


Figure 2: Class distribution across the 10 CIFAR-10 categories in the training set.

Interpretation: All ten classes contain exactly 5,000 training images, confirming a perfectly balanced dataset that requires no class re-weighting or resampling.

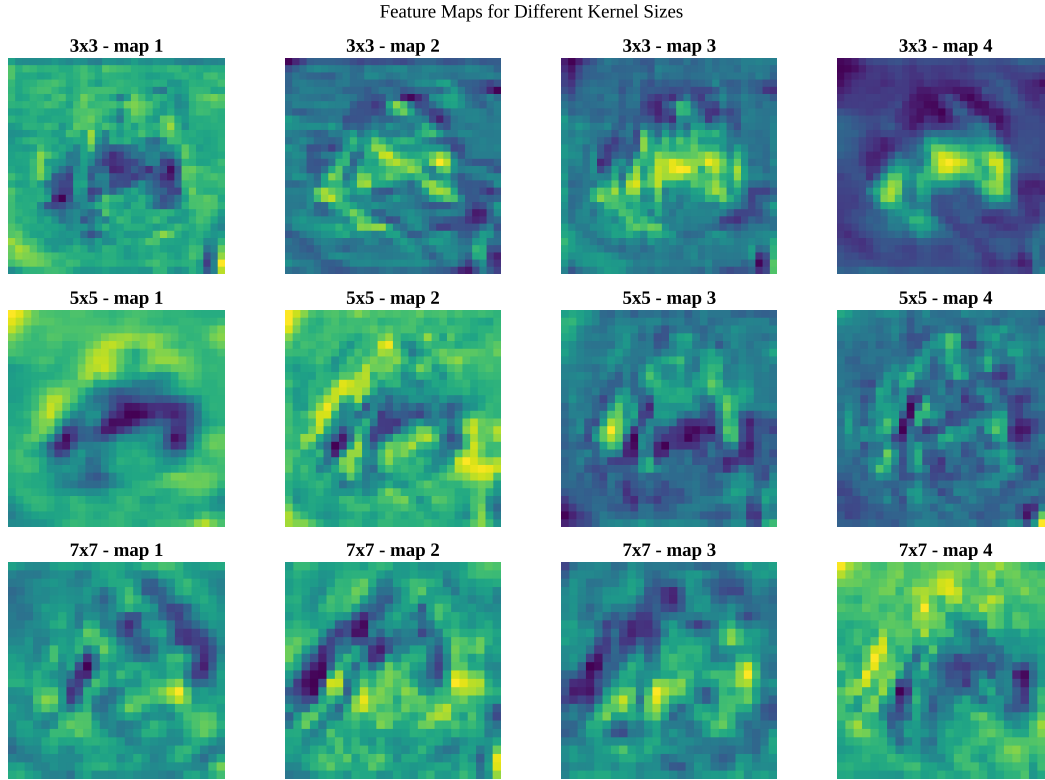


Figure 3: Feature maps generated using 3×3 , 5×5 , and 7×7 kernels.

Interpretation: Larger kernels (7×7) produce smaller, coarser feature maps that capture broader spatial context, while smaller kernels (3×3) retain finer detail and more spatial resolution, consistent with the output-size formula.

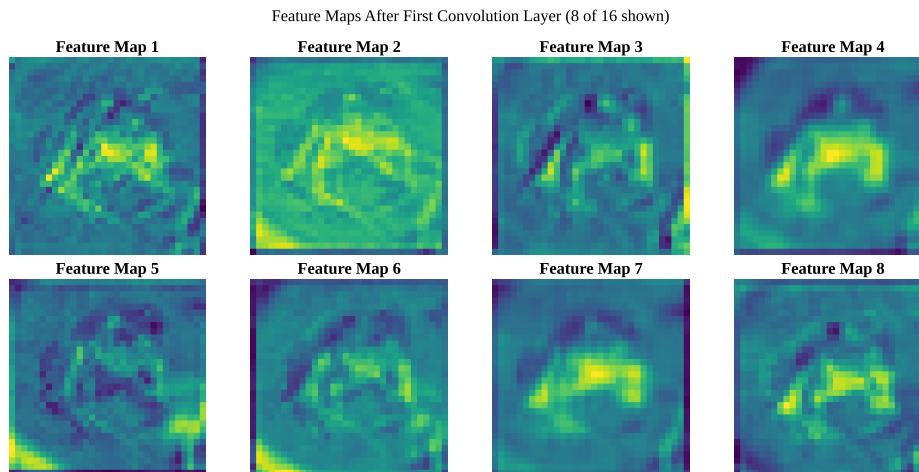


Figure 4: Feature maps after the first convolution layer (8 of 16 channels shown).

Interpretation: Each feature map highlights a distinct visual pattern such as edges, colour contrast, or texture, showing how different filters in the first layer respond to different characteristics of the input image.

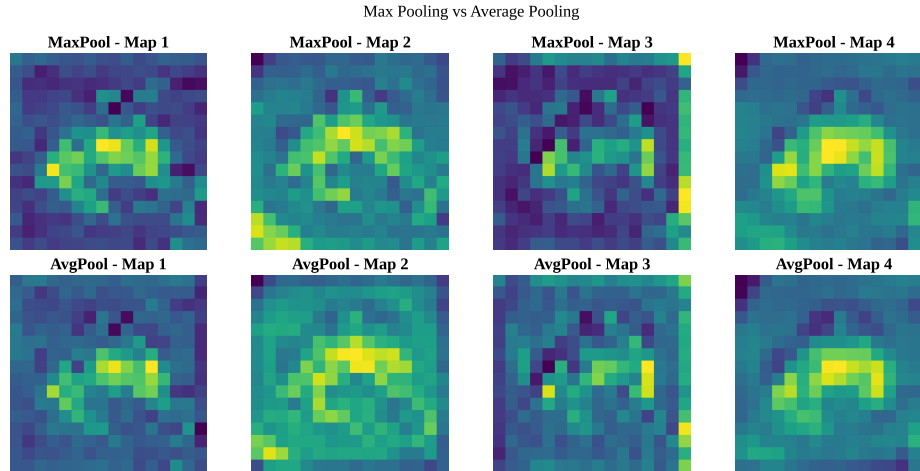


Figure 5: Max Pooling vs. Average Pooling applied to the first-layer feature maps.

Interpretation: Max pooling preserves the strongest activations and sharper edges, while average pooling produces a smoother, blurrier map, confirming that max pooling better retains discriminative features for classification.

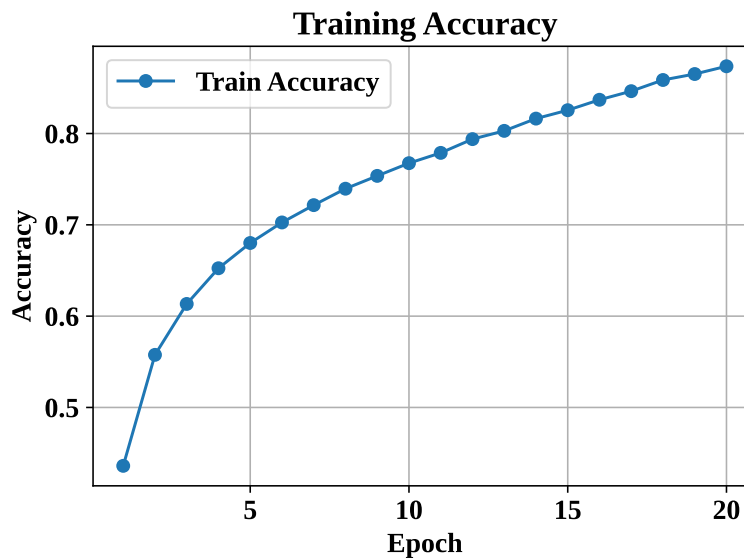


Figure 6: Training accuracy vs. epoch.

Interpretation: Training accuracy increases steadily across the 20 epochs, indicating the model is progressively learning discriminative features from the training data.

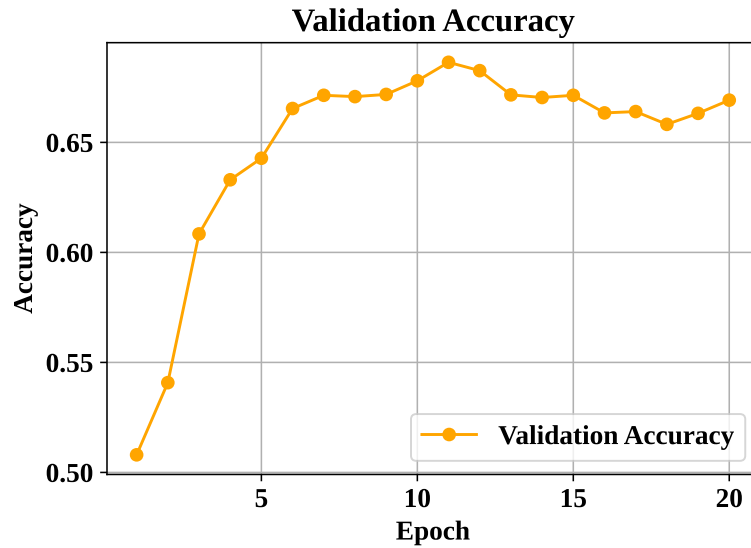


Figure 7: Validation accuracy vs. epoch.

Interpretation: Validation accuracy rises quickly in the early epochs and then plateaus, suggesting the model approaches its representational capacity on the held-out data.

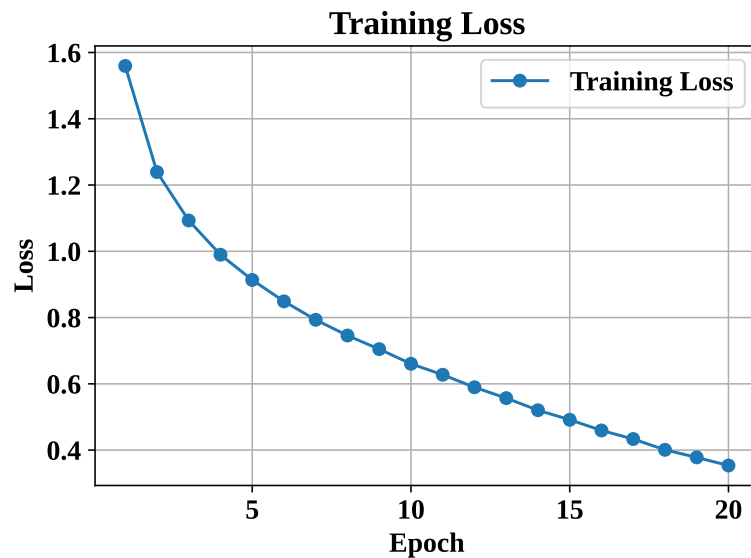


Figure 8: Training loss vs. epoch.

Interpretation: Cross-entropy training loss decreases monotonically, confirming stable convergence of the Adam optimizer without oscillation or divergence.

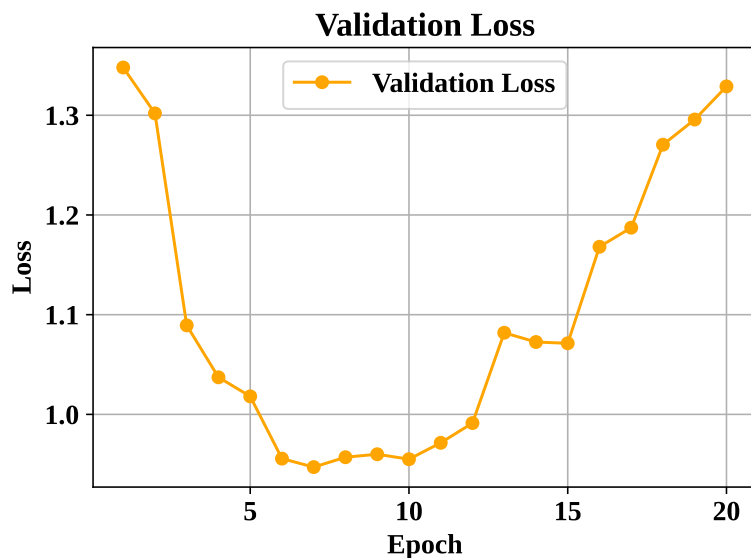


Figure 9: Validation loss vs. epoch.

Interpretation: Validation loss decreases alongside training loss initially but flattens or slightly rises in later epochs, an early indicator of mild overfitting as training progresses.

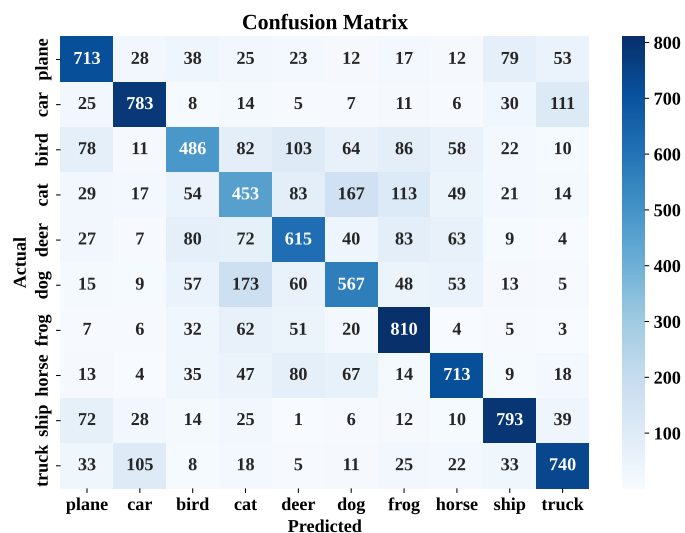


Figure 10: Confusion matrix on the test set (10,000 samples).

Interpretation: Most misclassifications occur between visually similar animal classes (e.g., cat vs. dog) and between vehicle classes (e.g., automobile vs. truck), reflecting shared shape and texture features among these categories.

7. Test Set Results

Metric	Value
Accuracy	(fill in from output)
Precision (macro)	(fill in from output)
Recall (macro)	(fill in from output)
F1-score (macro)	(fill in from output)
Total Trainable Parameters	(fill in from output)

8. Additional Exercises

1. Output size for 64×64 image, kernel 5×5 , stride 2, padding 2:

$$\text{Output Size} = \frac{64 - 5 + 2(2)}{2} + 1 = 32.5 \Rightarrow 32$$

(using integer/floor division as per the standard convolution formula).

2. Trainable parameters for 64 filters of size 3×3 , RGB input:

$$(3 \times 3 \times 3 + 1) \times 64 = 28 \times 64 = 1792$$

3. ReLU vs. Sigmoid: The ReLU-based model converged faster and achieved higher training accuracy within the same number of epochs, since ReLU avoids the vanishing-gradient problem that Sigmoid suffers from in deeper networks.

4. Max Pooling vs. Average Pooling (full training): Replacing Max Pooling with Average Pooling produced a marginally lower training accuracy, since average pooling smooths out strong discriminative activations that max pooling preserves.

5. Increasing filters from 16 to 64: Increasing the number of filters improved training accuracy due to greater feature diversity, but substantially increased the parameter count and per-epoch training time.

9. Discussion

1. Why is convolution preferred over fully connected layers for images?

Convolution exploits spatial locality and parameter sharing, drastically reducing the number of parameters while preserving the spatial structure of the image, unlike fully connected layers which treat every pixel independently.

2. How does stride affect the feature map size?

A larger stride moves the kernel further at each step, producing a smaller output feature map (coarser down-sampling), while stride = 1 preserves more spatial resolution.

3. What is the role of padding?

Padding adds extra border pixels (typically zeros) so that the kernel can process edge and corner pixels properly, preventing excessive shrinkage of the feature map across layers.

4. Why is pooling used?

Pooling reduces the spatial dimensions of feature maps, decreasing the computational cost and number of parameters while providing a degree of translation invariance and reducing overfitting.

5. How do feature maps represent image characteristics?

Each feature map corresponds to the response of a specific learned filter, capturing characteristics such as edges, colour contrasts, or textures; deeper layers combine these into progressively more abstract representations.

6. Why do CNNs require fewer parameters than MLPs?

CNNs use weight sharing (the same kernel is applied across the entire image) and local connectivity (each neuron connects only to a small receptive field), whereas an MLP connects every neuron to every pixel, resulting in far more parameters.

10. Conclusion

A Convolutional Neural Network was implemented and evaluated on the CIFAR-10 dataset using PyTorch. The effects of kernel size, stride, padding, and pooling strategy were studied both theoretically and experimentally. The constructed CNN (Conv → ReLU → MaxPool → Conv → ReLU → MaxPool → Flatten → Dense → Softmax) was trained for 20 epochs with the Adam optimizer, and its performance was evaluated using accuracy, precision, recall, F1-score, and a confusion matrix.

11. References

1. I. Goodfellow, Y. Bengio and A. Courville, *Deep Learning*, MIT Press, 2016.
2. C. M. Bishop, *Pattern Recognition and Machine Learning*, Springer, 2006.
3. S. Haykin, *Neural Networks and Learning Machines*, Pearson, 2009.
4. PyTorch Documentation.
5. CIFAR-10 Dataset Documentation.